Introduction to the Theory of Computation

Lecture 9. Turing Machine Programming

Neo (Yuan) Li

Oct 2026

Fudan University

yuan_li@fudan.edu.cn

Example: Palindrome

Decide if a 01 string is a palindrome, that is, if $w = w^R$.

For example:

Input: 0110 $\rightarrow$ **Accept**

Input: 0011 $\rightarrow$ **Reject**

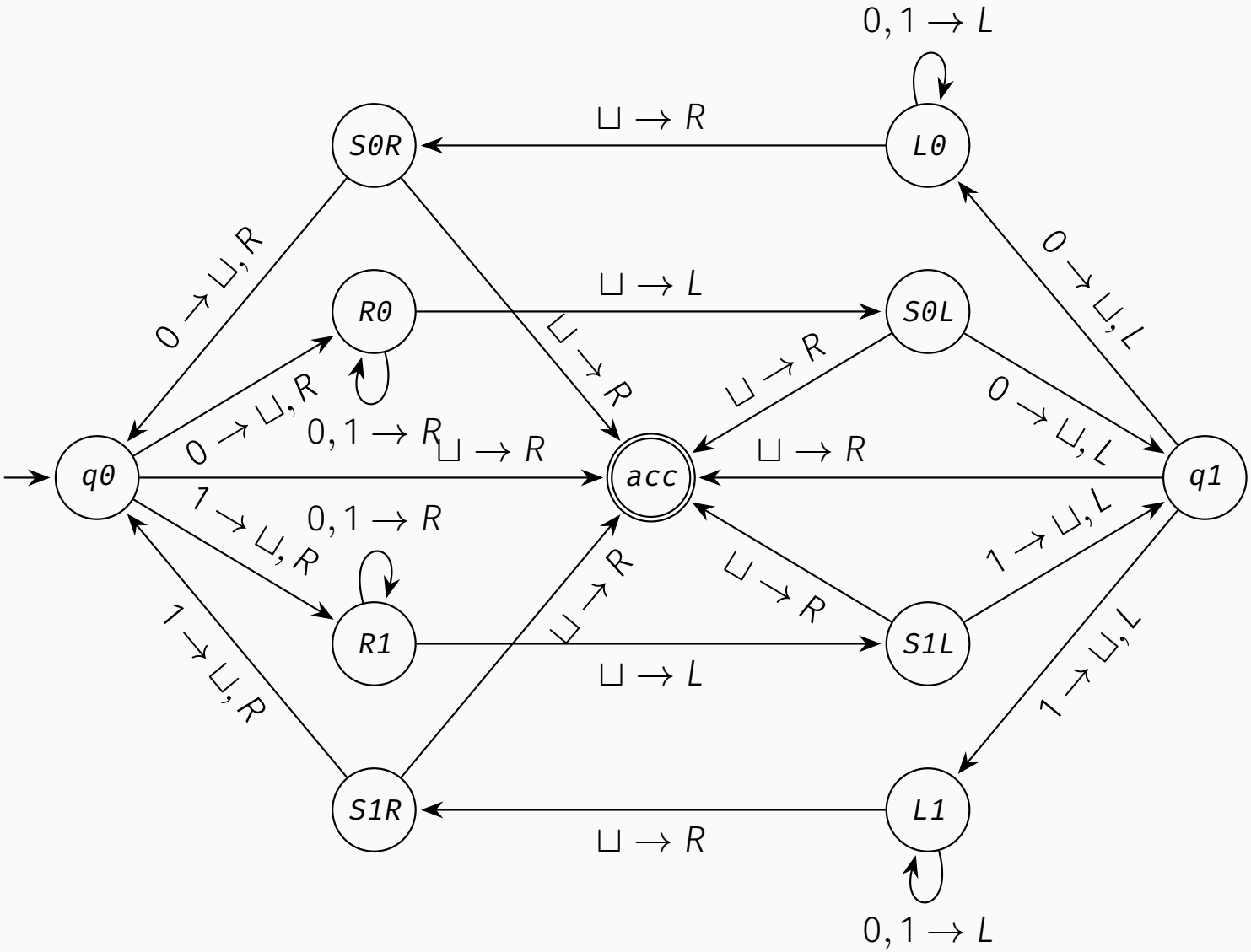
Input: 010 $\rightarrow$ **Accept**

The Idea: Compare Both Ends, Back and Forth

The head bounces back and forth between the two ends, peeling off one matching pair at a time:

1. Read the leftmost symbol, memorize it, and erase it.
2. Move to the rightmost end and read the symbol. If it does not exist, accept. If it does not match the leftmost (erased) one, reject. Otherwise, erase it.
3. Return to the current leftmost non-blank symbol and repeat.

A Turing Machine for Palindromes



Simulator code: *https://github.com/neoyuanli/fdu-toc-2026/blob/main/turing-machines/palindrome.txt*

Example: Deciding $L = \{0^{2^n} : n \geq 0\}$

Decide the language

$$L = \{\, 0^{2^n} \;:\; n \geq 0 \,\}.$$

That is, accept a string of 0's if and only if its length is a power of 2.

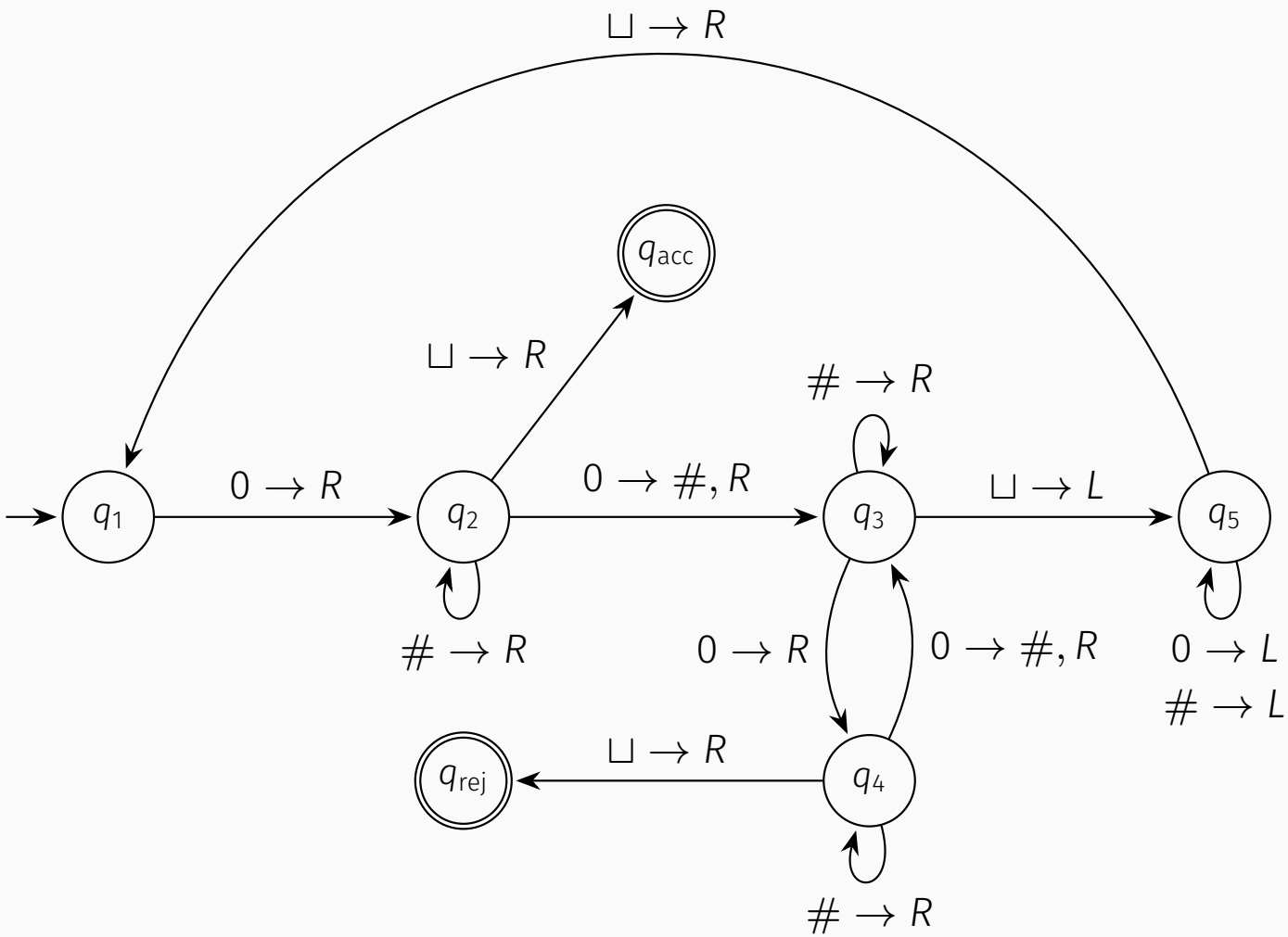
Input	Decision
0	Accept $(2^0 = 1)$
00	Accept $(2^1 = 2)$
0000	Accept $(2^2 = 4)$
00000000	Accept $(2^3 = 8)$
ϵ (empty string)	Reject
000	Reject
00000	Reject

The Idea: Repeatedly Halve the Count

On input $w \in \{0\}^*$:

1. Sweep from left to right, crossing out every second 0.
2. If exactly one 0 remains on the tape, **accept**.
3. If more than one 0 remains and their count is odd, **reject**.
4. Move the head back to the left end of the tape.
5. Repeat from Step 1.

A Turing Machine for $\{0^{2^n} : n \geq 0\}$



Simulator code: *https://github.com/neoyuanli/fdu-toc-2026/blob/main/turing-machines/power_of_two_zeros.txt*

Example: Binary Comparison

Compare two binary numbers x and y, separated by a comma; accept if $x > y$.

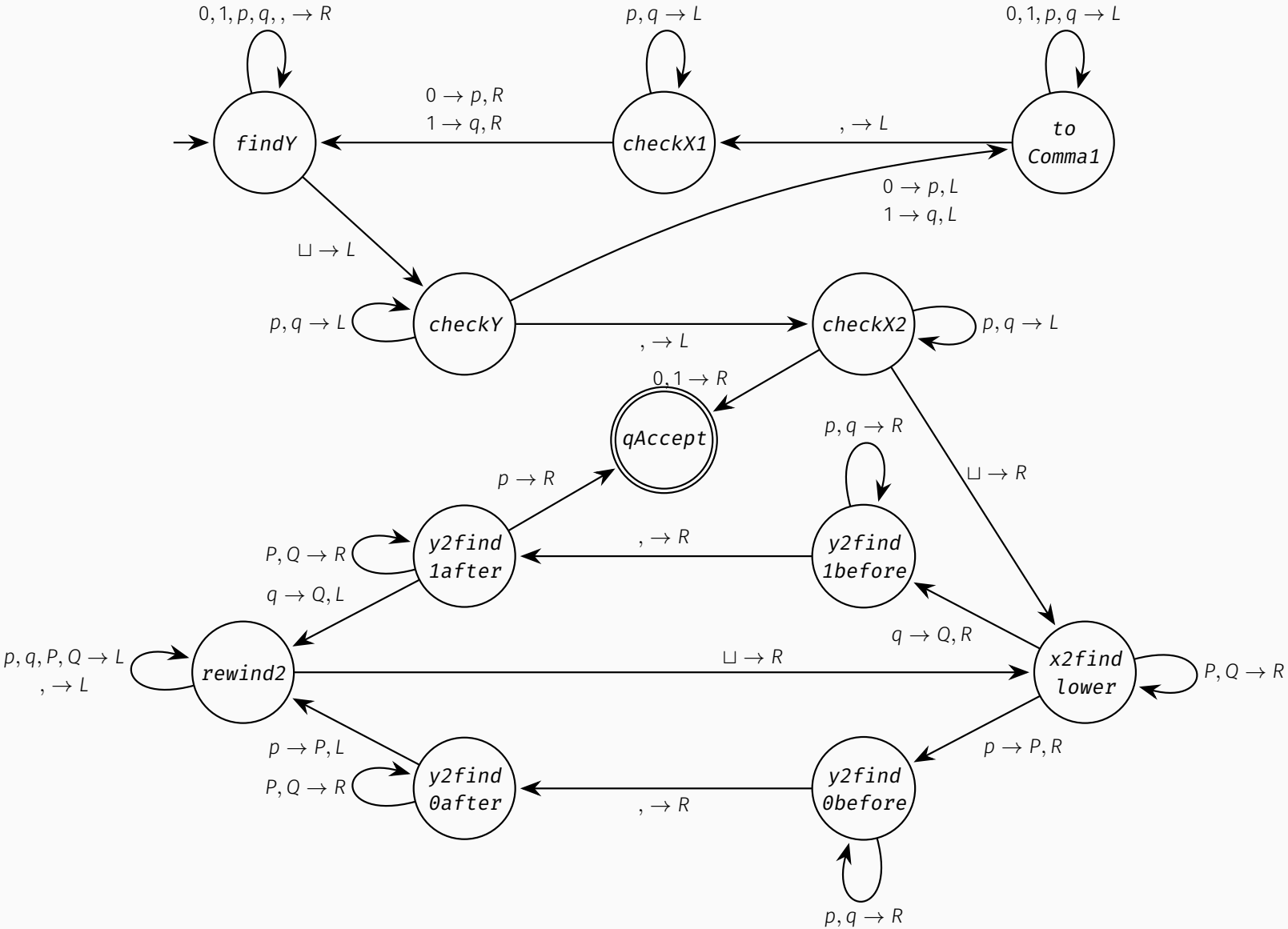
- **Input:** x, y (binary numbers, most significant bit first; no leading zeros unless the number is zero), written on the tape as x, then a comma, then y
- **Alphabet:** $\Sigma = \{0, 1, ,\}$

x	y	Decision	
101	11	Accept	$(5 > 3)$
11	101	Reject	$(3 < 5)$
101	101	Reject	$(5 = 5)$
1	0	Accept	$(1 > 0)$

The Idea: First Compare Lengths, Then Compare Values

1. **Phase 1 (compare lengths).** Repeatedly mark off the rightmost remaining digit of x together with the rightmost remaining digit of y — marking, not erasing — until one of them has no digits left. If same length, go to Phase 2.
2. **Phase 2 (same length, compare values).** Scan both numbers from the left, comparing digits pair by pair. The first position where they differ decides the answer.

Binary Comparison: State Diagram



Simulator code: *https://github.com/neoyuanli/fdu-toc-2026/blob/main/turing-machines/binary_comparison.txt*

Example: Binary Addition

- **Input:** $x + y$ (binary numbers, most significant bit first), written on the tape as x, then a plus sign, then y
- **Alphabet:** $\Sigma = \{0, 1, +\}$
- **Output:** $x + y$

x	y	$x + y$
101 (5)	1 (1)	110 (6)
111 (7)	1 (1)	1000 (8)
1 (1)	1 (1)	10 (2)

Binary Addition: The Idea

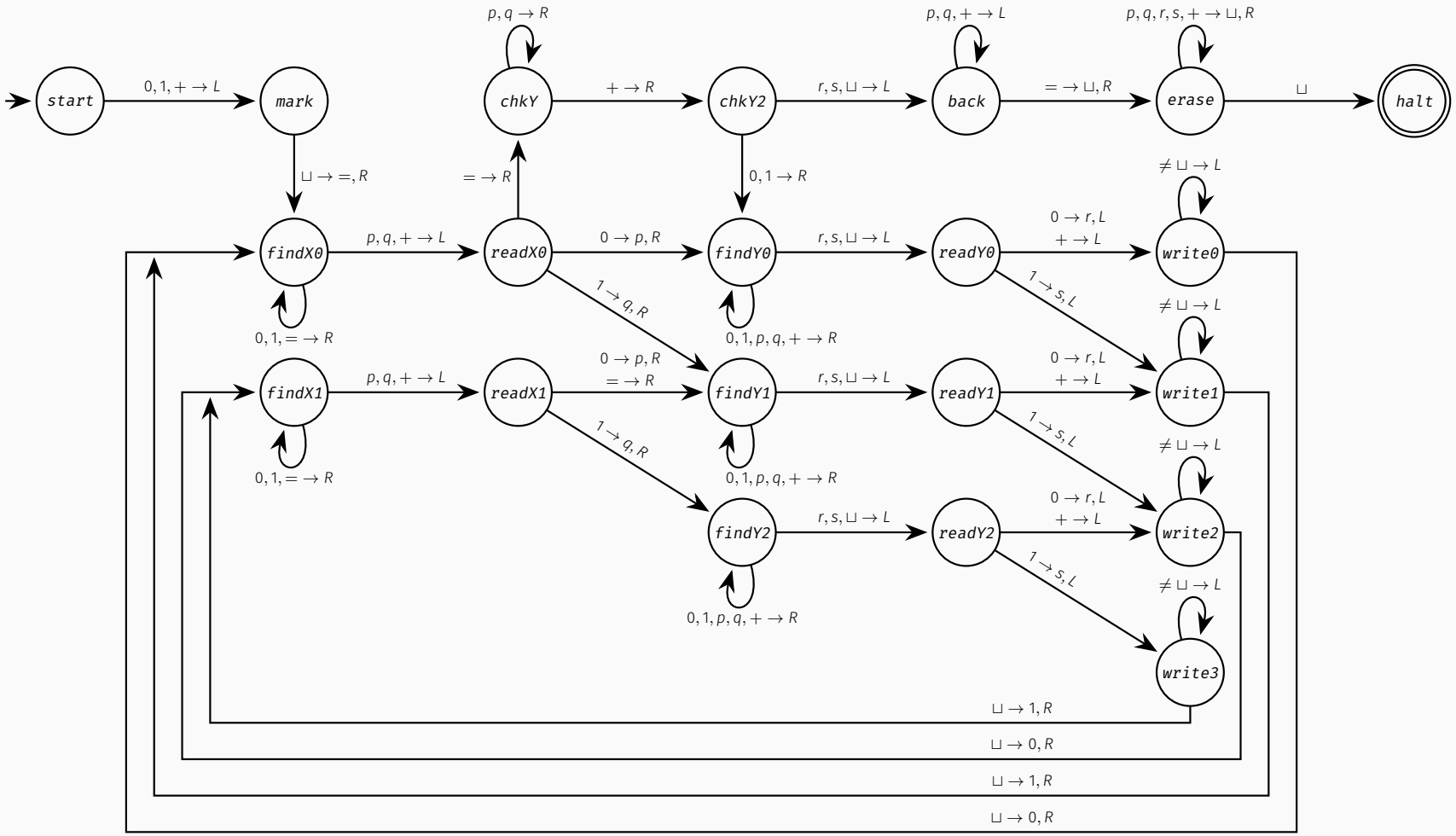
- **Tape layout:** $z = x+y$, with z initially empty
- **Work from the lowest bit.** In each round:
  1. read the lowest unused bit of x and of y (a missing bit counts as 0), and mark both as used
  2. compute $s = x_i + y_i + c$
  3. write $s \bmod 2$ **immediately to the left** of the bits of z written so far; remember the new carry $c = \lfloor s/2 \rfloor$ in the state
- **Stop** when x and y are used up (if $c = 1$, write a final 1), then **erase** everything except z.

Binary Addition: An Example

Example: $101 + 1 \;\; (5 + 1)$

Step	Tape	What happened
0	*=101+1*	start, $c = 0$
1	*0=10#+#*	$1 + 1 + 0 = 2$: write 0, $c = 1$
2	*10=1##+#*	$0 + 0 + 1 = 1$: write 1, $c = 0$
3	*110=###+#*	$1 + 0 + 0 = 1$: write 1, $c = 0$
4	*110*	all used, $c = 0$: erase *=###+#*

Binary Addition: State Diagram



Simulator code: *https://github.com/neoyuanli/fdu-toc-2026/blob/main/turing-machines/binary_addition.txt*

Example: Binary Multiplication

- **Input:** $x*y$ (binary numbers, most significant bit first), written on the tape as x, then a star, then y
- **Alphabet:** $\Sigma = \{0, 1, *\}$
- **Output:** $x \cdot y$, also most significant bit first

x	y	$x * y$
101 (5)	11 (3)	1111 (15)
11 (3)	11 (3)	1001 (9)
110 (6)	10 (2)	1100 (12)

Binary Multiplication: The Idea

- **Shift and add:**

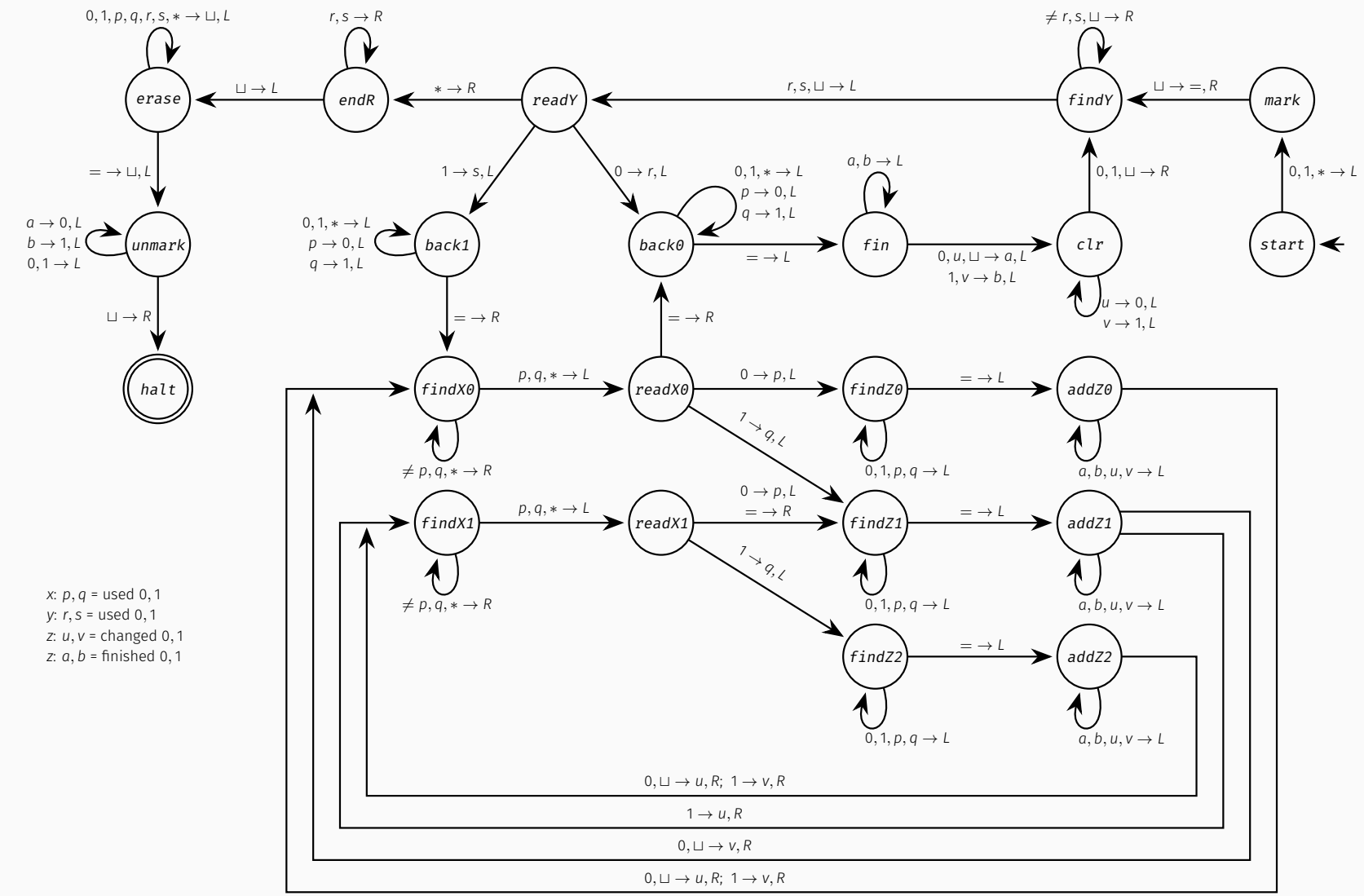
$$x \cdot y = \sum_{j\,:\,y_j=1} x \cdot 2^j$$

- **Tape layout:** $z = x * y$, with z initially empty
- **Key trick:** after handling y_j, the lowest $j+1$ bits of z never change again
  - mark them as **finished**
  - the finished marks play the role of the shift 2^j

Binary Multiplication: One Round

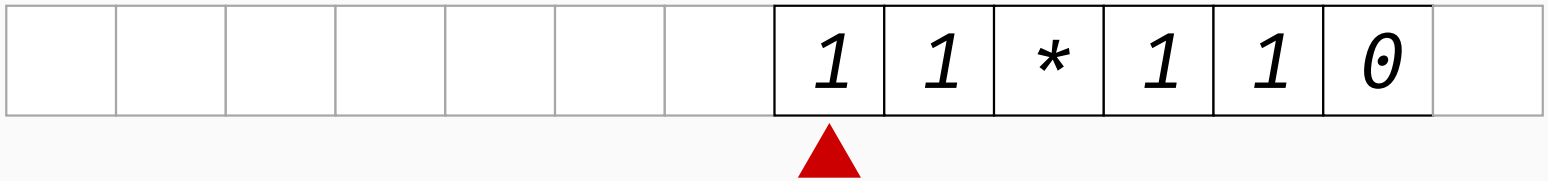
- **Work from the lowest bit of y.** In each round:
  1. read the lowest unused bit y_j of y; mark it as used
  2. if $y_j = 1$: add x into z **in place** , aligned at the lowest **unfinished** bit of z
  3. mark the lowest unfinished bit of z as **finished**
- **Stop** when y is used up; remove the marks on z and **erase** everything except z.

Binary Multiplication: State Diagram



Simulator code: *https://github.com/neoyuanli/fdu-toc-2026/blob/main/turing-machines/binary_multiplication.txt*

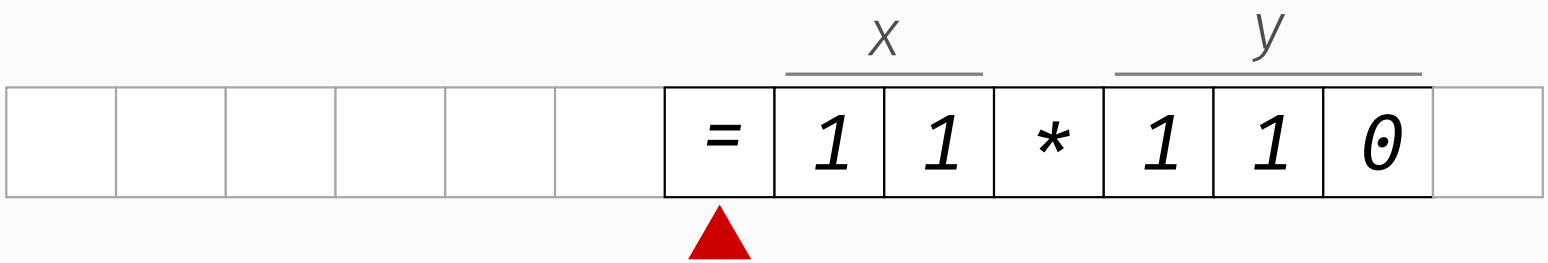
Example: 11×110 (3×6) — Setup



Input: $x * y$ $(x = 11, y = 110)$

used bit of x or y | bit of z changed this round | finished bit of z | head

Example: 11×110 (3×6) — Setup

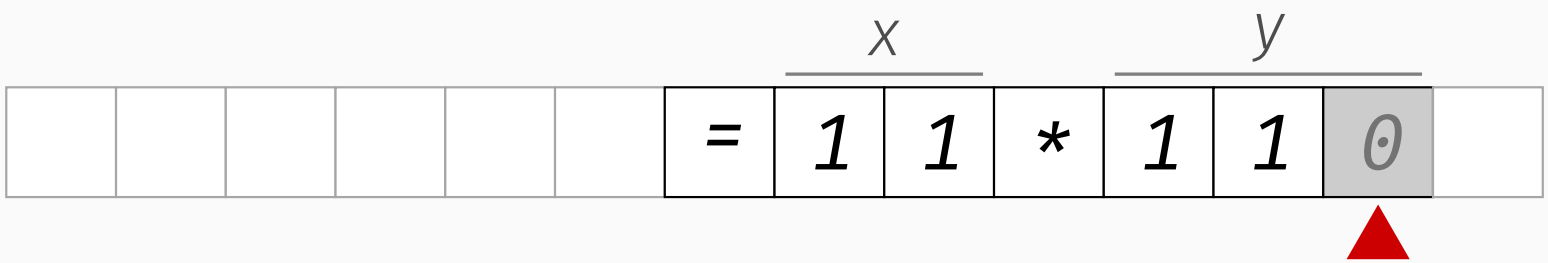


Write = to the left of x.

The result z (now empty) will grow to the left of =.

used bit of x or y | bit of z changed this round | finished bit of z | head

Example: 11×110 (3×6) — Round 1: $y_0 = 0$

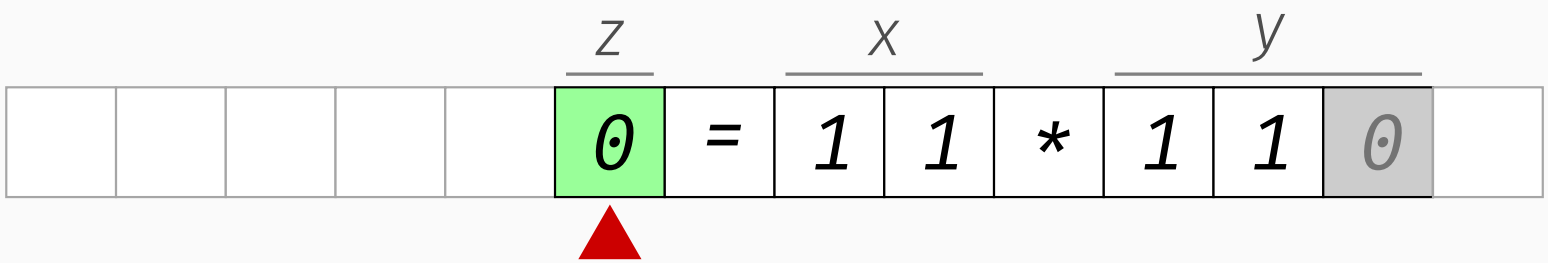


Read the lowest unused bit of y: $y_0 = 0$. Mark it as used.

It is 0: nothing to add.

used bit of x or y | bit of z changed this round | finished bit of z | head

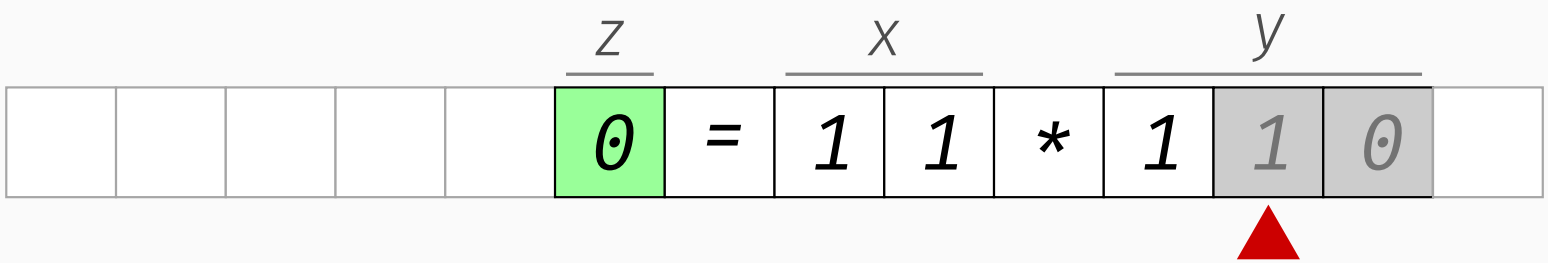
Example: 11×110 (3×6) — Round 1: $y_0 = 0$



The lowest unfinished bit of z will never change again: mark it as **finished** (it is blank, so write a 0).

used bit of x or y | bit of z changed this round | finished bit of z | head

Example: 11×110 (3×6) — Round 2: $y_1 = 1$

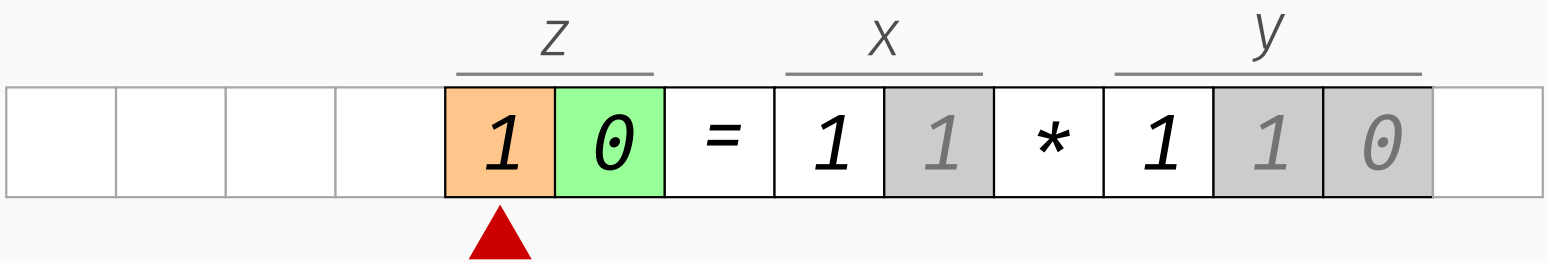


Read the lowest unused bit of y: $y_1 = 1$. Mark it as used.

It is 1: add x into z, starting at the lowest unfinished bit.

used bit of x or y | bit of z changed this round | finished bit of z | head

Example: 11×110 (3×6) — Round 2: $y_1 = 1$



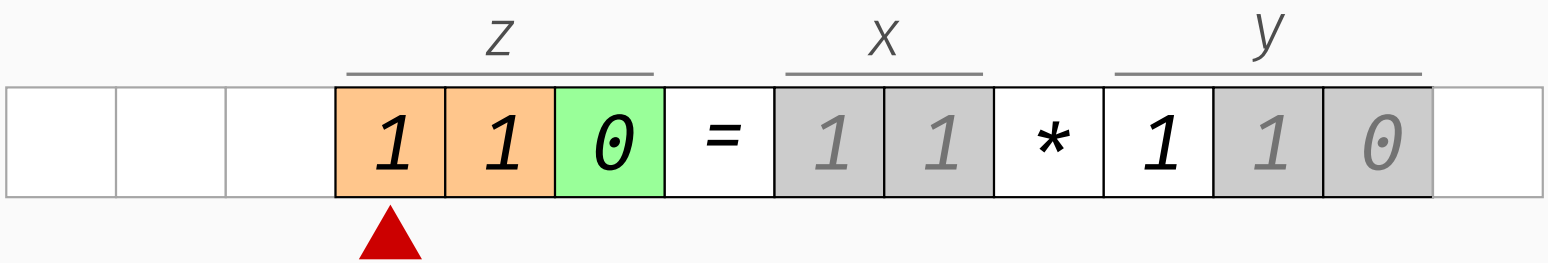
Add the next bit of x into z:

z bit $+$ x bit $+$ carry $= 0 + 1 + 0 = 1$ (blank counts as 0)

$\Rightarrow$ write 1, carry $= 0$

used bit of x or y | bit of z changed this round | finished bit of z | head

Example: 11×110 (3×6) — Round 2: $y_1 = 1$



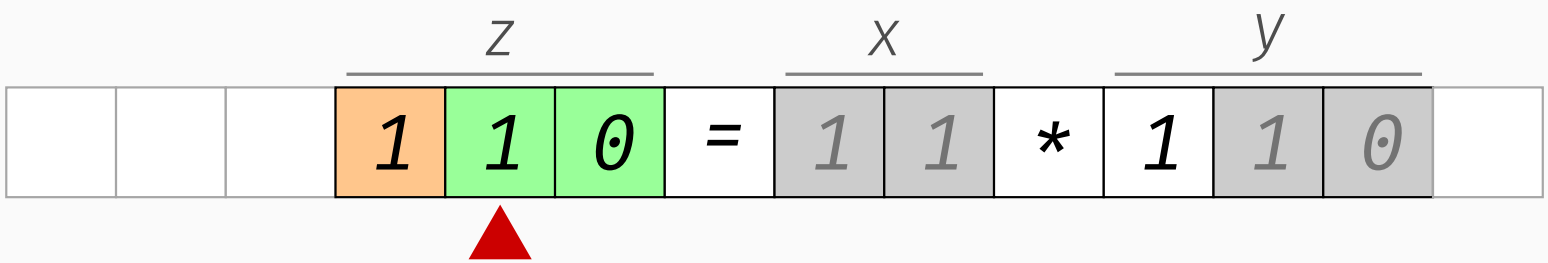
Add the next bit of x into z:

z bit $+$ x bit $+$ carry $= 0 + 1 + 0 = 1$ (blank counts as 0)

$\Rightarrow$ write 1, carry $= 0$

used bit of x or y | bit of z changed this round | finished bit of z | head

Example: 11×110 (3×6) — Round 2: $y_1 = 1$



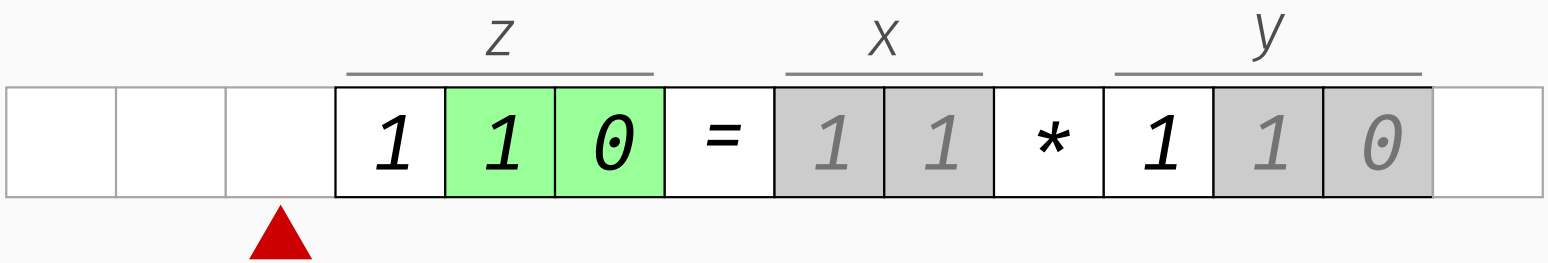
x is used up and carry $= 0$: the addition is done.

The lowest unfinished bit of z will never change again:

mark it as **finished**.

used bit of x or y | bit of z changed this round | finished bit of z | head

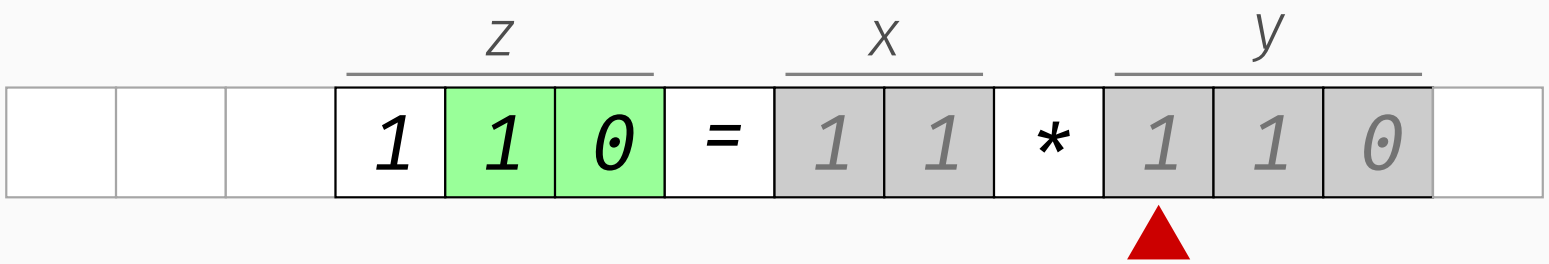
Example: 11×110 (3×6) — Round 2: $y_1 = 1$



Clear this round's marks on z. The round is done.

used bit of x or y | bit of z changed this round | finished bit of z | head

Example: 11×110 (3×6) — Round 3: $y_2 = 1$

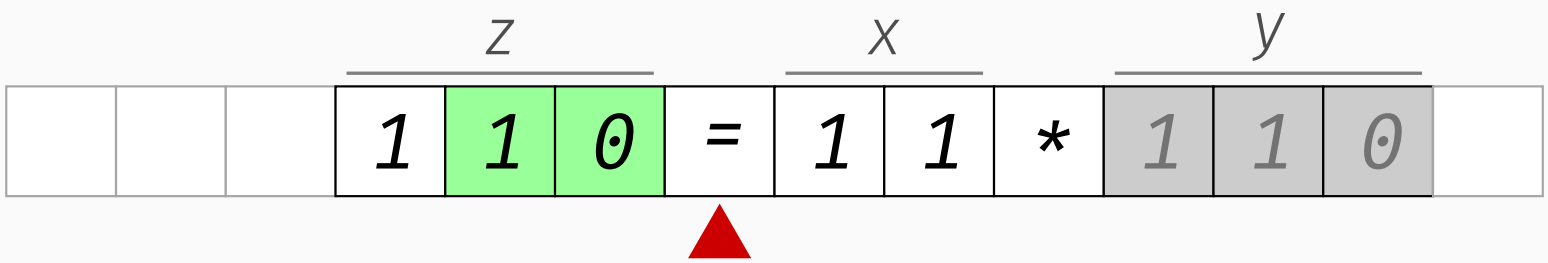


Read the lowest unused bit of y: $y_2 = 1$. Mark it as used.

It is 1: add x into z, starting at the lowest unfinished bit.

used bit of x or y | bit of z changed this round | finished bit of z | head

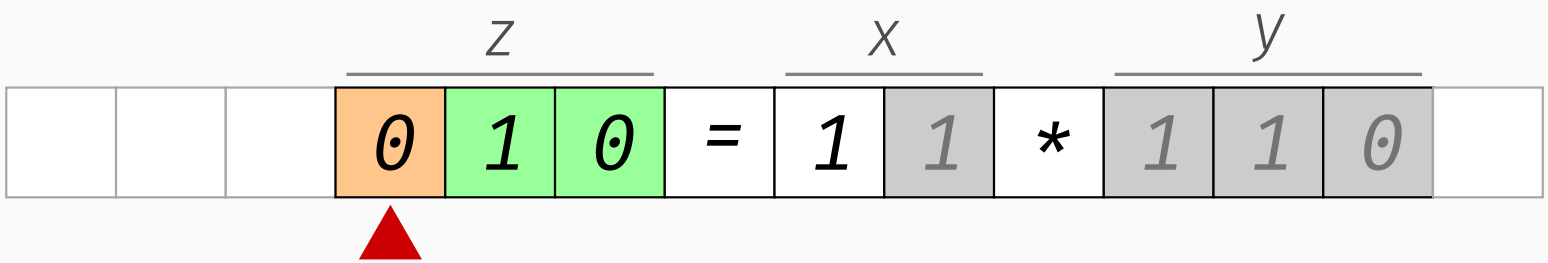
Example: 11×110 (3×6) — Round 3: $y_2 = 1$



Go back to =, restoring the bits of x (we need x again).

used bit of x or y | bit of z changed this round | finished bit of z | head

Example: 11×110 (3×6) — Round 3: $y_2 = 1$



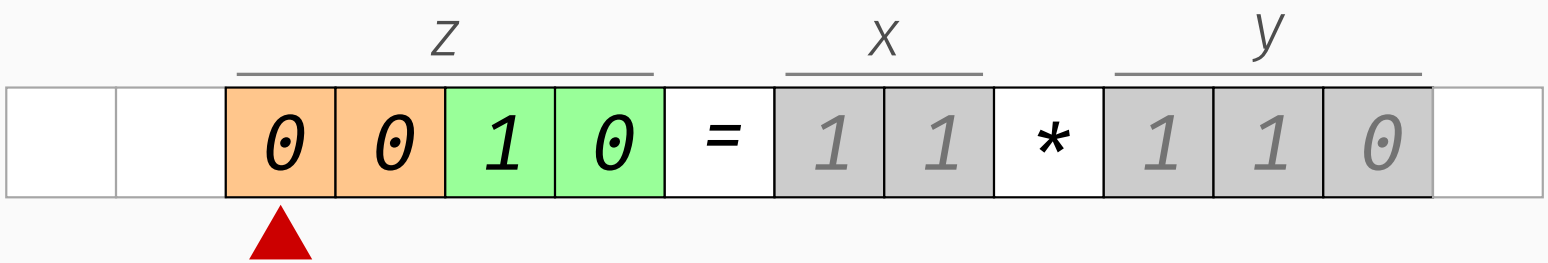
Add the next bit of x into z:

z bit $+$ x bit $+$ carry $= 1 + 1 + 0 = 2$

$\Rightarrow$ write 0, carry $= 1$

used bit of x or y | bit of z changed this round | finished bit of z | head

Example: 11×110 (3×6) — Round 3: $y_2 = 1$



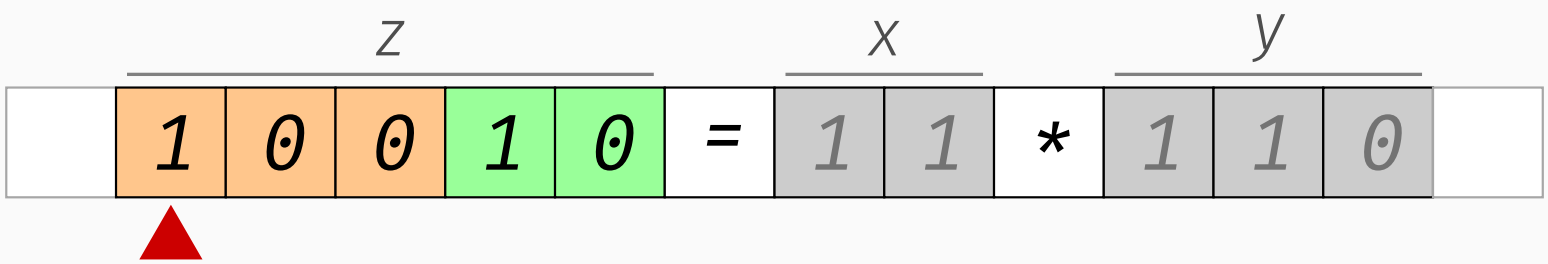
Add the next bit of x into z:

z bit $+$ x bit $+$ carry $= 0 + 1 + 1 = 2$ (blank counts as 0)

$\Rightarrow$ write 0, carry $= 1$

used bit of x or y | bit of z changed this round | finished bit of z | head

Example: 11×110 (3×6) — Round 3: $y_2 = 1$



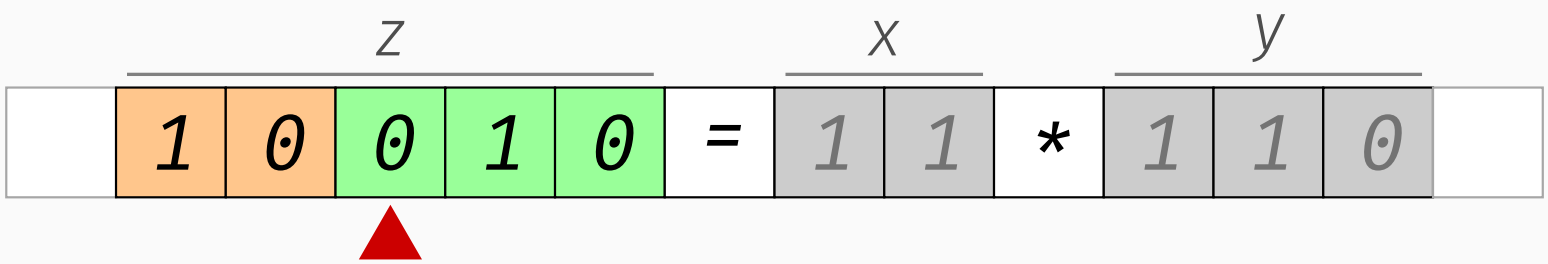
x is used up, but carry $= 1$:

z bit $+$ carry $= 0 + 1 = 1$ (blank counts as 0)

$\Rightarrow$ write 1, carry $= 0$

used bit of x or y | bit of z changed this round | finished bit of z | head

Example: 11×110 (3×6) — Round 3: $y_2 = 1$



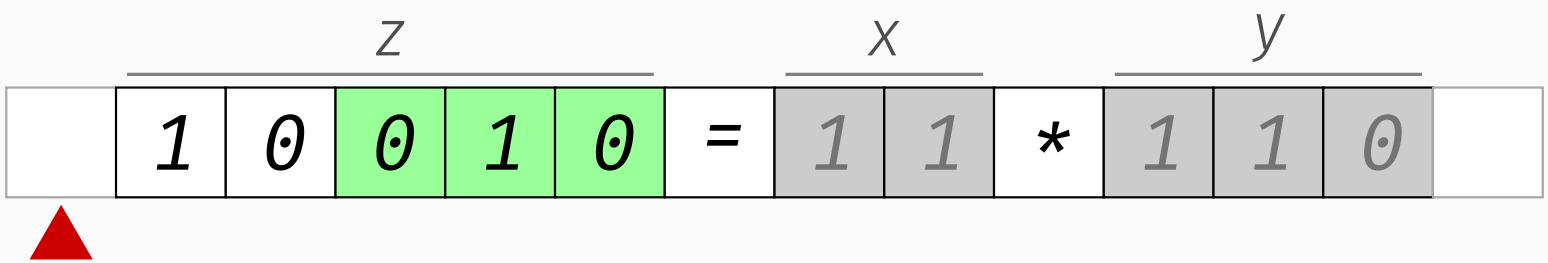
x is used up and carry $= 0$: the addition is done.

The lowest unfinished bit of z will never change again:

mark it as **finished**.

used bit of x or y | bit of z changed this round | finished bit of z | head

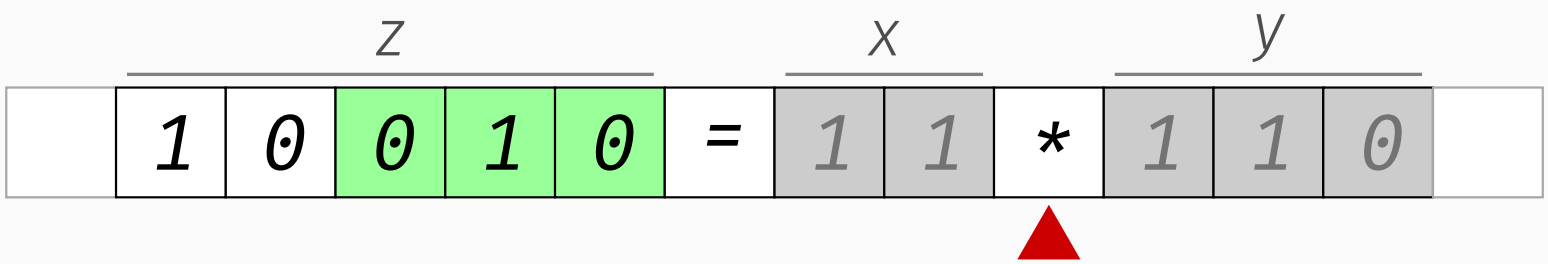
Example: 11×110 (3×6) — Round 3: $y_2 = 1$



Clear this round's marks on z. The round is done.

used bit of x or y | bit of z changed this round | finished bit of z | head

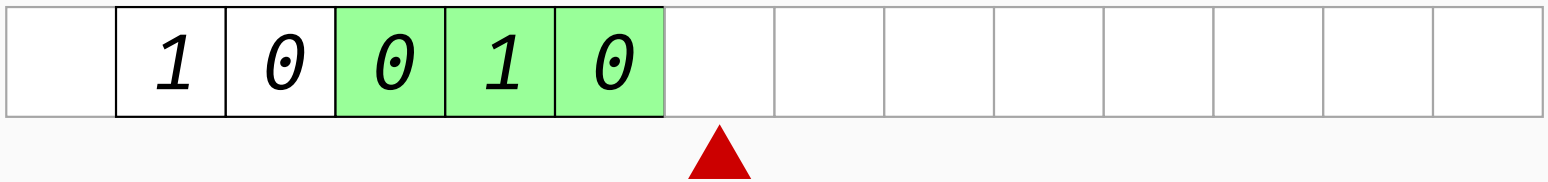
Example: 11×110 (3×6) — Finish



Reached $*$: y is used up.

used bit of x or y | bit of z changed this round | finished bit of z | head

Example: 11×110 (3×6) — Finish



Erase = and everything to its right.

used bit of x or y | bit of z changed this round | finished bit of z | head

Example: 11×110 (3×6) — Finish



Remove the marks on z.

Result: $z = 10010$ $(3 \times 6 = 18)$.

used bit of x or y | bit of z changed this round | finished bit of z | head